AlgoCamp AI Engineering Notes — Speculative Decoding, Temperature & LLM Engineering Approaches

> *AI Engineering cohort, session 3. Notes on LLM API providers & pricing, speculative decoding (drafter + target, the three-step process, Meta's Muse Glimmer / DFlash example), the temperature parameter (what it actually does), and the three LLM-engineering paradigms: prompt engineering, context engineering (RAG), and harness engineering.*

---

1. LLM API Providers & Pricing

Where you actually get model access as an AI engineer (extends session 2's provider discussion):

- **OpenAI** — GPT models; first-party API, pay-per-token.
- **Anthropic** — Claude models; first-party API, pay-per-token.
- **Groq** — ultra-fast inference hardware (LPUs); low per-token latency. *(Groq the inference provider — not xAI's "Grok" chatbot; the recap's "Grok" refers to Groq here.)*
- **OpenRouter** — a **router/aggregator**: one API key, many models across providers, easy switching and price comparison. (What **Task Buddy** routes through.)

Pricing & free tiers

Most providers offer **free tiers** for developers to experiment, plus paid pay-as-you-go. As noted last session, free/cheap tiers can mean lower-quality outputs, rate limits, or smaller model variants. Cost is billed per token (input + output), so context length and output length directly drive your bill.

---

2. Speculative Decoding — The Big Idea

The problem it solves

Recall from session 2: LLMs are **autoregressive** — they generate **one token at a time**, sequentially, each depending on the last. For a large model, every single token requires a full forward pass through billions of parameters. That's slow, especially for long outputs.

The insight

Most tokens are *easy to predict* — a small, fast model can guess them correctly most of the time. So: let a **cheap model draft**, and only use the **expensive model to check**.

```
Traditional:   BIG model    token    BIG model    token    BIG model    token ...
               (one expensive pass PER token — slow)

Speculative:   small DRAFTER proposes several tokens at once
               BIG TARGET model verifies them ALL in ONE pass
               accept the correct ones, in bulk
```

The two models

- **Drafter** (small, fast) — proposes multiple candidate tokens quickly and cheaply.
- **Target** (large, accurate) — verifies the drafts in a single parallel pass, accepting the good ones and correcting the first wrong one.

The magic: verifying N tokens in **one** target-model pass is far cheaper than generating N tokens in **N** target passes. When the drafter is right (which is often), you get several tokens for the price of roughly one big-model step.

---

3. The Three-Step Process

AlgoCamp broke speculative decoding into three phases:

Step 1 — Draft (probability calculation)

The drafter generates a block of candidate tokens, each with the probability *it* assigned.

```
Drafter proposes:  "the" (0.9)    "cat" (0.8)    "sat" (0.7)    "on" (0.85)
```

Step 2 — Parallel verification

The target model processes **all** the drafted tokens in a **single forward pass**, computing the probability *it* would have assigned to each. This is the key efficiency win — one pass checks many tokens.

> *A subtle but important point AlgoCamp made: the target (transformer) model can compute "what probability would I have given this specific token here?" for each drafted position at once. That's what makes bulk verification possible — it's checking, not generating from scratch.*

Step 3 — Rejection (probability comparison)

Compare drafter vs target probabilities per token:

```
For each drafted token:
  target agrees (prob high enough)     ACCEPT
  target disagrees                     REJECT here, target supplies the correct token,
                                       discard the rest of the draft, restart drafting
```

Accepted tokens are locked in together; the first rejection is corrected by the target; drafting resumes from there.

Why quality is preserved

Because the **target model has the final say** on every token, the output is **statistically equivalent to what the target would have produced alone** — you get its quality, just faster. Speculative decoding is a *speed* optimization, not a *quality* trade-off. (The research paper — *"Fast Inference from Transformers via Speculative Decoding"* — proves this output-equivalence; students should read it for the concept, not the heavy math.)

---

4. Real Example — Meta's Muse Glimmer & DFlash

The live demo used **Muse Glimmer**, Meta's 30-billion-parameter open-weight model (released August 2026, Apache 2.0, distilled from Meta's larger Muse Spark). It's a good real-world anchor because it ships speculative decoding built in.

Muse Glimmer's drafter is called **DFlash** — a lightweight companion network that **proposes entire blocks of 16 tokens** (one anchor + 15 proposed) in a single pass, which the main 30B model then **verifies in parallel**, accepting correct tokens and correcting wrong ones. Meta reports the output is **mathematically identical** to normal token-by-

token generation — exactly the quality-preserving property from §3 — while decoding **substantially faster** (Meta cites roughly a 3× decode speedup on high-end GPUs).

It also ties back to session 2's hardware themes: 4-bit **quantization** shrinks the 30B model to run on a single ~24 GB consumer GPU (or a Mac's unified memory), with the DFlash drafter adding only small memory overhead — so speculative decoding + quantization together are what make a capable model run *fast* on *modest* hardware.

Memory/cost angle

Speculative decoding is generally **cheaper in time and compute**: fewer expensive big-model passes per token. The cost is a **little extra memory** (you load the small drafter alongside the target) and some added system complexity. For most workloads the speed win dwarfs the memory overhead.

> ***A correction worth making for students:*** *the recap raised "why aren't big companies like OpenAI using this?" — in reality,* ***speculative decoding is widely used in production*** *across the industry (OpenAI, Anthropic, Google, and open stacks like vLLM all employ it or similar techniques). It's a standard, mature inference optimization, not an untested startup trick. The startup-vs-enterprise analogy is a fine point about robustness at scale, but on the specific technique: the big labs very much do use it.*

---

5. Temperature — What It *Actually* Does

The common misconception

People say "higher temperature = more creative." That's the *effect*, but not the *mechanism*. **Temperature doesn't add creativity — it flattens the probability distribution.**

The mechanism

Recall the model outputs a probability distribution over the next token via **softmax** (session 1). Temperature **T** scales the logits *before* softmax:

```
softmax_with_temp(logit_i) = e^(logit_i / T) / Σ e^(logit_j / T)
```

```
Low T (  0):   divides logits by a small number    SHARPENS the distribution
                  the top token dominates    near-deterministic, focused
High T (>1):  divides logits by a big number    FLATTENS the distribution
                  lower-probability tokens get a real chance    varied, surprising
```

Visually:

```
T = 0.2 (sharp):                                  almost always the top token
T = 1.0 (normal):
T = 1.8 (flat):                                   unlikely tokens become plausible
```

Higher temperature just means **the less-likely tokens become more likely to be picked** — which *reads* as more creative/surprising, but it's really distribution flattening. Crucially, temperature **doesn't change how many tokens are generated** — only *which* ones get selected.

The OpenAI SDK

In the OpenAI SDK, **temperature ranges 0 to 2** (default ~1):

```
response = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.2,     # focused/factual   ...   up to 2 = wild/creative
)
```

Use-case guidance

Task	Temperature	Why
Code generation	~0 – 0.2	want correct, deterministic, no surprises
Factual Q&A / extraction	~0	want the single best answer, repeatable
Summarization	0.3 – 0.7	mostly faithful, a little fluency
Marketing copy	0.8 – 1.2	want variety and flair
Poetry / story / brainstorm	1.0 – 2.0	want surprise and range

As **IneshReddy** summarized in class: keep it **very close to 0 for factual tasks**, use **1 and above for creative storytelling.** Good rule of thumb.

> *The cricket example: AlgoCamp used cricket to show how external factors shift prediction probabilities — a reminder that the distribution isn't fixed; context (and temperature) reshapes which token wins. For coding you want the sharp, "obvious next token" behavior; for a poem you want the flattened distribution that lets unexpected words through.*

---

6. Where Knowledge Lives — Parameters, Not a Database (recurring clarification)

When IneshReddy asked about "probability values in token databases," AlgoCamp reiterated a key point: **LLMs store information in their parameters (weights), not in a database.** There is no lookup table of tokens and probabilities being queried. The knowledge is compressed into billions of weights during training — analogous to how human knowledge lives in the brain's neural connections, not a filing cabinet. (Same clarification as session 1 — worth repeating because it's a persistent beginner intuition to unlearn. Vector databases, by contrast, are a separate *application-layer* thing for RAG — §7.)

---

7. The Three LLM Engineering Approaches

A framing for *how* you build with LLMs — the ladder from simple to sophisticated:

1. PROMPT ENGINEERING	craft the instruction well
2. CONTEXT ENGINEERING	control what information the model sees (incl. RAG)
3. HARNESS ENGINEERING	build a whole ecosystem around the model

1. Prompt Engineering

Getting the most out of a model by **how you phrase the request** — clear instructions, examples (few-shot), format specifications, chain-of-thought prompts. The cheapest, first lever. (Few-shot and chain-of-thought are the next lectures.)

2. Context Engineering (including RAG)

Controlling **what information is in the context window** (session 2, §3–4) so the model has what it needs. **RAG (Retrieval-Augmented Generation)** is the headline technique: retrieve relevant documents (often from a **vector database**) and inject them into the context at query time, so the model answers from *your* data, not just its training.

```
User question  retrieve relevant docs (vector DB)  put them in context  model answe
```

This is where vector databases actually belong (§6) — retrieval, not model storage.

3. Harness Engineering

The most advanced: building a **complete ecosystem around the LLM** — not just prompting it, but wrapping it in:

- **Tools** — function calling, APIs, code execution.
- **Evaluation mechanisms** — automated checks on output quality (LLM-as-judge, test suites).
- **Sub-agents** — specialized agents that handle parts of a complex task.

The harness turns a raw model into a **system** that can tackle real, multi-step work — coding assistants, legal-document analysis, autonomous agents. The model is one component; the harness is everything that makes it reliable and capable end-to-end.

```
                    HARNESS
        tools · evals · sub-agents · memory

        input        LLM        output
```

> *This ladder maps the cohort's trajectory: you start with prompts, add context/RAG, and build up to harnesses/agents. Muse Glimmer being "built for agents" (§4) is a sign of where the field is heading — models increasingly assume a harness around them.*

What's Coming

- Lectures on **few-shot prompting** and **chain of thought** this week.
- The **speculative decoding paper** (*Fast Inference from Transformers via Speculative Decoding*) + a couple of articles.

- **Local setup** videos (Ollama / LM Studio) on Discord.
- Possibly an **earlier neural-network intro** to solidify parameters/math intuition.
- Next session: build a **small agent**, or discuss the **prompt engineering + templating engines (Jinja)** paper.

Action Items

AlgoCamp:

- Share Ollama/LM Studio local-setup videos on Discord / dashboard.
- Share the speculative decoding paper + articles.
- Update the lecture title to include “speculative decoding” and “temperature.”
- Consider an earlier neural-network primer.
- Next session: small agent, or the prompt-engineering/Jinja templating paper.

Concept Cheat-Sheet

- **Providers:** OpenAI · Anthropic · Groq (fast inference) · OpenRouter (multi-model router, used by Task Buddy); free tiers = lower quality; billed per token.
- **Speculative decoding** small **drafter** proposes several tokens, large **target** verifies them in one parallel pass faster, **same quality**.
- **3 steps:** draft (drafter’s probs) parallel verify (target’s probs, one pass) reject/accept (compare; target corrects the first miss).
- **Quality preserved** because the target has final say output equivalent to target-alone, just faster.
- **Muse Glimmer / DFlash** Meta’s 30B open model + its 16-token-block drafter; identical output, ~3× faster; runs on a 24GB GPU at 4-bit.
- **Speculative decoding is standard in production** — widely used by major labs, not experimental.
- **Temperature** scales logits before softmax; **low = sharp/focused**, **high = flat/varied**. It *flattens the distribution*, doesn’t “add creativity.” Doesn’t change token count.
- **OpenAI temp range 0–2**; ~0 for code/facts, 1+ for creative (IneshReddy’s rule).
- **Knowledge lives in weights**, not a database (brain analogy); vector DBs are for RAG retrieval.
- **3 engineering approaches:** prompt engineering context engineering (RAG, vector DBs) harness engineering (tools + evals + sub-agents = a full system around the model).